

TRINETRA AI

Automated Photo Identification and Classification for Traffic Violations Using Computer Vision

Team TRINETRA AI

Gridlock Round 2 – Submission

June 21, 2026

Agenda

- 1 Problem
- 2 Architecture
- 3 Prototype Demo
- 4 Coverage & Roadmap
- 5 Conclusion

The Problem

- Traffic surveillance cameras generate **huge volumes** of images daily.
- Manual review for violations is **slow, costly, and inconsistent**.
- Human inspection cannot scale with growing camera networks.
- Need: an **automated, AI-driven pipeline** that can detect vehicles, classify violations, read plates, and generate enforceable evidence – with zero manual intervention.

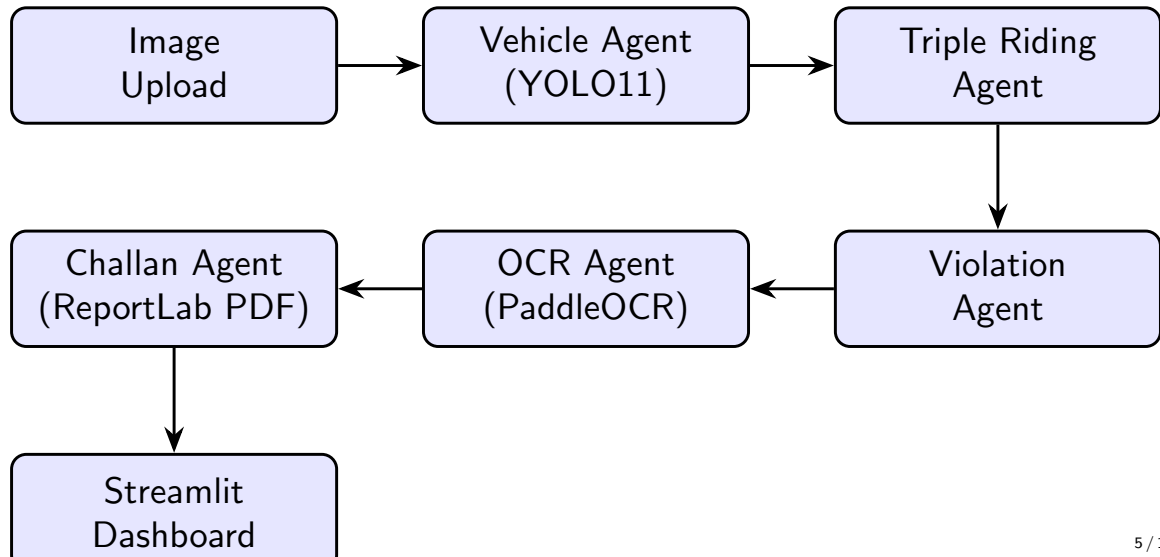
TRINETRA AI

A multi-agent computer vision pipeline that turns a single traffic scene image into a fully-formed, official PDF challan – automatically.

One-line idea

Detect → Identify Violation → Read Plate → Generate Evidence → Issue Challan – all in one pipeline, one image, no human in the loop.

System Architecture – Multi-Agent Pipeline



What Each Agent Does

Agent	Role
Vehicle Agent	YOLO11 detects persons, motorcycles, cars, buses, trucks
Triple Riding Agent	Flags violation if motorcycles ≥ 1 & persons ≥ 3
Violation Agent	Maps detected conditions $\rightarrow$ violation labels
OCR Agent	PaddleOCR reads license plate (full image, then per-vehicle crop)
Challan Agent	Builds PDF ticket with ReportLab: plate, time, evidence image

Stub agents (designed, planned for expansion): helmet_agent, tracking_agent, enhancement_agent, plate_agent

Component	Library / Tool
Object Detection	Ultralytics YOLO11
License Plate OCR	PaddleOCR + PaddlePaddle
PDF Generation	ReportLab
Web Dashboard	Streamlit (dark mode)
Image Processing	OpenCV, Pillow
DL Backend	PyTorch, PaddlePaddle (CPU/GPU auto-switch)

Demo – Dashboard UI

← → ↺

🔒 trinetraai.streamlit.app

📄 70% ☆

📄 📄 📄 📄 ☰

drag and drop or browse a file, select image...

 Screens...232836.png 171.0KB +

Image uploaded successfully! Ready for processing.

TRINETRA AI

Smart Traffic Enforcement System

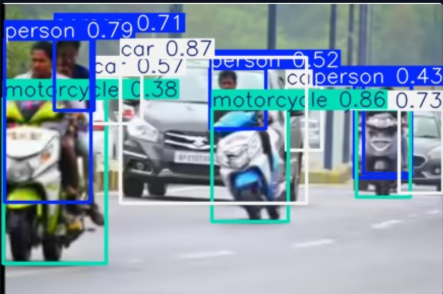
System Information

- 🔌 Status: Operational
- ⚙️ Engine: YOLO11 + PaddleOCR
- 🔄 Mode: Real-time Single Image Inference

Trinetra AI monitors traffic feeds and automatically generates challans for violations such as Triple Riding.

AI Detection Analysis

🔍 Annotated Evidence 🖼️ Original Image



person 0.79 0.71
car 0.87
person 0.52
car 0.43
motorcycle 0.38
motorcycle 0.86 0.73

Enforcement Metrics

VIOLATION RISK INDEX

60%

MOTORCYCLES

4

PERSONS

5

EXTRACTED PLATE NUMBER

NO PLATE DETECTED

Violation Assessment

⚠️ Triple Riding DETECTED

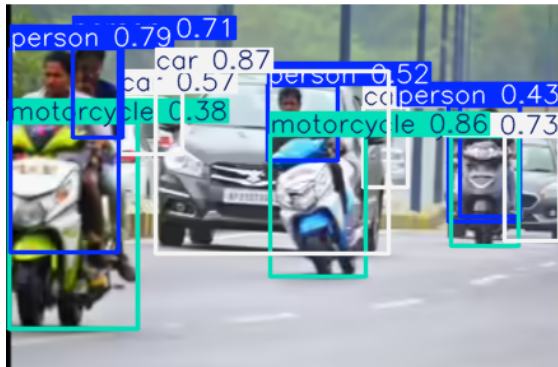
Manage app

TRINETRA AI - Detections and Class Labels

Demo – Detection & Annotated Evidence



Original Image



YOLO Detection (Annotated)

Violation Detection Logic

Current Prototype: Triple Riding

Condition: $\text{motorcycle_count} \geq 1$ **AND** $\text{person_count} \geq 3$ on the same motorcycle $\Rightarrow$ flagged as **Triple Riding**.

Architecture is rule-based and **pluggable** – new violation types slot in as new agents without touching the core pipeline:

- Helmet non-compliance
- Seatbelt non-compliance
- Wrong-side driving
- Stop-line / red-light violation
- Illegal parking

Two-pass ANPR strategy:

- 1 Run PaddleOCR on the **full image**, regex-match Indian plate format XX00XX0000 (e.g. DL01AB1234).
- 2 If no match → crop each detected vehicle's bounding box and re-run OCR **per crop** to boost accuracy.

CPU by default; auto-switches to GPU if CUDA-enabled PaddlePaddle is available.

Violation Risk Score

Condition	Points
Triple Riding detected	+40
Each additional violation	+20
Maximum score	100 (capped)

Risk score surfaces directly on the dashboard as a single metric authorities can use to triage and prioritize review.

[SCREENSHOT]

Generated PDF challan: receipt no., plate, violations, embedded evidence image

Contains: Receipt Number, Vehicle Plate, Date & Time, Violations List, Embedded Evidence Image.

Mapping to Challenge Requirements

Challenge Task	Status
Vehicle / road user detection & classification	Implemented
Triple riding detection	Implemented
License plate detection & OCR	Implemented
Evidence generation (annotated image + metadata)	Implemented
PDF challan / reporting output	Implemented
Helmet non-compliance	Planned (stub agent ready)
Seatbelt non-compliance	Planned
Wrong-side driving	Planned
Stop-line / red-light violation	Planned (needs multi-frame tracking)
Illegal parking	Planned (needs multi-frame tracking)
Image enhancement (low light / rain / blur)	Planned (stub agent ready)

Scalability & Future Roadmap

- **Agent isolation** keeps compute efficient – OCR Agent only fires when Vehicle Agent confirms a vehicle exists.
- Deployable on **edge devices** or scaled as **cloud microservices** for thousands of camera feeds.
- Next agents to activate: `helmet_agent`, `tracking_agent` (multi-frame, enables red-light/parking/wrong-side), `enhancement_agent` (super-resolution for low light/rain/blur), `plate_agent` (dedicated plate-region detector for higher OCR accuracy).
- Evaluation plan: **mAP** for detection, **Precision / Recall / F1** for violation classification, latency & throughput for scalability.

Expected Impact

- Removes manual bottleneck in reviewing surveillance footage.
- Standardizes ticketing – consistent rules, no human bias.
- Produces a ready-to-issue PDF challan with embedded proof, end to end.
- Modular design means new violation types are additive, not a rewrite.

Thank You

Questions?

github.com/Almaas-Chaudhary/gridlock_round_2